

Gestion de projet agile (MSPR3 / TPRE601)

Mise en production de la plateforme HealthAI Coach issue des MSPR1 et MSPR2. Ce document décrit la manière dont le travail d'industrialisation a été organisé : méthodologie, découpage en sprints, cérémonies, suivi Kanban et gestion des risques.

1. Methodologie

L'approche retenue est agile et itérative, adaptée à la taille réelle de l'équipe et au périmètre de la MSPR3. Le principe : livrer par incréments fonctionnels, chacun vérifiable de bout en bout (la stack démarre, les nouveaux éléments sont démontrables), plutôt que de tout produire en une seule passe.

Contexte concret :

- Petite équipe. Pas de processus lourd : les cérémonies sont allégées et le pilotage se fait directement dans GitHub (issues, pull requests, Actions).
- L'application mobile (mini réseau social) est prise en charge par un collègue. Elle est donc hors du périmètre de ce livrable : aucune tâche de ce document ne la concerne.
- La cible de mise en production est un déploiement local orchestré par Docker Compose (`bootstrap.sh`), point d'évaluation et de démonstration. Le découpage en sprints est pensé autour de cette cible.

Quatre incréments (sprints) ont été réellement suivis, dans cet ordre :

Sprint	Theme	Etat
A	Observabilité	Termine
B	Configurations multi-environnement + résilience	Termine
C	CI/CD et qualité	Termine
D	Documentation, sécurité et gestion de projet	Termine

2. Decoupage en sprints

Sprint A : Observabilité

Objectif. Doter la plateforme d'un système de supervision : savoir si les services sont vivants, suivre leurs ressources, centraliser les logs, être alerté en cas de problème.

Principales tâches.

- Overlay `docker-compose.monitoring.yml` ajoutant la stack d'observabilité sans modifier la stack applicative : Prometheus, Grafana, Loki, Promtail, Alertmanager.
- Exporteurs d'infrastructure : cAdvisor (conteneurs), node-exporter (hôte), postgres-exporter x2 (bases métier et auth), mongodb-exporter.

- Instrumentation applicative des services dans leur code source : API via Actuator / Micrometer (`/api/actuator/prometheus`), AI-Nutrition et Reco-Fitness via prometheus-fastapi-instrumentator (`/metrics`), Auth via `@hono/prometheus` (`/metrics`).
- Trois regles d'alerte Prometheus (`prometheus/alerts.yml`) : CibleInjoignable (`up == 0` pendant 1 min, critical), `ConteneurCpuEleve` (> 0.9 coeur CPU pendant 5 min, warning), `ConteneurMemoireElevee` (> 2 Go de RAM pendant 5 min, warning).
- Dashboard Grafana "MSPR HealthAI - Vue stack" provisionne automatiquement (cibles UP, disponibilite par job, CPU / memoire par conteneur, flux de logs).

Livrables. Overlay monitoring fonctionnel, instrumentation des 4 services, alertes, dashboard provisionne, documentation `monitoring/README.md` (composants + liste exhaustive des donnees collectees).

Sprint B : Configurations multi-environnement et resilience

Objectif. Rendre la stack adaptable a plusieurs contextes de demonstration et capable de sauvegarder / restaurer ses donnees.

Principales taches.

- Trois configurations via overlays Compose, documentees dans `CONFIGS.md` :
 - **complete** : tous les services + monitoring, IA generative active (`docker-compose.yml` + `docker-compose.monitoring.yml`).
 - **offline** : sans internet (`docker-compose.offline.yml`), LLM force sur Ollama local, Auth sans verification email (`AUTH_OFFLINE=true` , aucun appel Resend).
 - **performance** : limites CPU / RAM par service (`docker-compose.performance.yml`), IA generative lourde omise du demarrage par default (repli sur la matrice statique), monitoring reduit a l'essentiel.
- Scripts de resilience dans `scripts/` operant sur les conteneurs en cours : `backup.sh` (dump horodate : `pg_dump x2 + mongodump`), `restore.sh` (`pg_dump --clean` , `mongorestore --drop`), `clean.sh` (remise a zero des volumes).

Livrables. Trois overlays Compose operationnels, `CONFIGS.md` , trois scripts de sauvegarde / restauration / nettoyage, section dediee dans le `README.md` .

Sprint C : CI/CD et qualite

Objectif. Industrialiser : chaque depot construit, teste et publie automatiquement son image, avec des garde-fous de qualite.

Principales taches.

- Une chaine GitHub Actions par depot (8 services) : lint selon la techno, tests + couverture, build de l'image Docker, publication sur GHCR (`ghcr.io/whitefoxyt/mspr-<service>`). Detail dans `CICD.md` .
- Publication conditionnee : uniquement sur `push` (pas sur les pull requests) et seulement si lint et tests passent. Les workflows `docker-publish` sont appeles via `workflow_call` .

- Couverture imposee a 80 % cote API (JaCoCo) et Reco-Fitness (pytest-cov).
- Strategie de tags via `docker/metadata-action` (branche, semver sur tags `vX.Y.Z`, SHA court, `latest` depuis la branche par default).
- Etape SonarCloud sur l'API, declenchee automatiquement des que le secret `SONAR_TOKEN` est present (ignoree sinon, sans casser le pipeline). L'API embarque aussi un environnement Sonar local (`docker-compose.sonar.yml`) pour une analyse manuelle.
- Durcissement des conteneurs (volet securite operationnelle de l'industrialisation) : les 6 services applicatifs tournent en utilisateur non-root, `HEALTHCHECK` presents, `no-new-privileges` et rotation des logs json-file (`max-size: 10m` , `max-file: 3`) sur tous les services, ports des bases (5433 / 5434 / 27018) bindes sur `127.0.0.1`.

Livrables. 8 pipelines CI/CD publiant sur GHCR, seuils de couverture configures, etape SonarCloud cote API, `docker-compose.yml` durci, `CICD.md`.

Sprint D : Documentation, securite et gestion de projet

Objectif. Finaliser les livrables redactionnels, consolider la gestion des secrets et formaliser le suivi projet.

Principales taches.

- Documentation de deploiement : `README.md` (prerequis, demarrage en une commande, acces aux services, depannage), `CONFIGS.md`, `CICD.md`, `monitoring/README.md`.
- Gestion des secrets : `bootstrap.sh` genere `BETTER_AUTH_SECRET` (`openssl rand`) et les mots de passe des bases (plus de default "password" en exploitation) ; les cle API (Resend, Mistral) sont chiffrees dans `secrets/*.enc` (`openssl aes-256-cbc, pbkdf2`), dechiffrees a la volee ; `.env` hors versionnement.
- Rappel du socle RGPD herite des MSPR precedentes : datasets sources anonymises, table `users` supprimee (migration V7), pas d'identifiant commun ni de PII dans les donnees chargees.
- Redaction de ce document de gestion de projet.

Livrables. Jeu de documents `MSPR-Deploy/` a jour, mecanisme de secrets en place, present document.

3. Ceremonies

Les ceremonies Scrum classiques sont conservees dans leur intention mais allagees pour une petite equipe :

Ceremonie	Adaptation
Planification de sprint	En debut de sprint : selection des issues GitHub du sprint, definition de l'objectif (le theme A / B / C / D) et du "fini" (la stack démarre et le nouvel increment est demonstrable).
Point d'avancement	Informel et asynchrone, via les commentaires d'issues et l'etat des pull requests / runs Actions. Pas de daily formel vu la taille de l'equipe.
Revue de sprint	Demonstration de l'increment sur la stack locale : lancement de l'overlay concerne, verification (<code>docker compose ps</code> , endpoints de sante, dashboard Grafana, run CI vert).
Retrospective	Bilan court en fin de sprint : ce qui a fonctionne, points de friction (ex : duree de pull d'Ollama, instrumentation visible seulement apres rebuild des images), ajustements pour le sprint suivant.

4. Tableau Kanban et outils

Le suivi des taches s'est fait sur un Kanban simple a trois colonnes, materialise par les issues GitHub et leur etat :

A faire	En cours	Termine
Issues planifiees du sprint, non demarrees	Issue en cours de developpement, pull request ouverte	Pull request mergee, increment verifie sur la stack

Outils :

- **GitHub Issues** : decoupage du travail, regroupement par sprint (theme A / B / C / D).
- **Pull requests** : une PR par lot de travail ; merge apres tests verts. Sert de point de revue.
- **GitHub Actions** : CI/CD par depot (lint, tests, couverture, build, publication GHCR). L'etat des runs fait office d'indicateur de qualite continu.
- **GHCR** : registre d'images (`ghcr.io/whitefoxyt/mspr-<service>`), source du mode prod.

5. Gestion des risques

Approche simple : identifier les risques concrets rencontres et la parade mise en place.

Risque	Impact	Parade
Ollama (gemma3:4b) lourd en CPU / RAM	Stack lente ou instable sur materiel modeste	Configuration performance qui omet <code>ai-nutrition</code> + <code>ollama</code> par default (repli sur la matrice statique) et applique des limites CPU / RAM ; alerte <code>ConteneurCpuEleve</code> .
Dependance a internet (pull d'images, modele Ollama, emails Resend, API Mistral)	Demo impossible hors-ligne	Configuration offline : Ollama local, <code>AUTH_OFFLINE=true</code> (sans Resend), prerequis telecharges une fois avec internet.
Premier pull tres long (Ollama ~3 Go)	Demarrage initial lent	Documente dans le <code>README.md</code> (depannage), suivi via <code>docker compose logs -f ollama</code> .
Gestion des secrets	Fuite de cle / mot de passe par default	Secrets generes par <code>bootstrap.sh</code> (<code>openssl rand</code>), cle API chiffrees dans <code>secrets/*.enc</code> (aes-256-cbc), <code>.env</code> hors versionnement, bases bindees sur <code>127.0.0.1</code> .
Metriques applicatives invisibles	Supervision incomplete	Documente : l'instrumentation vit dans le code source, donc visible seulement apres rebuild (mode dev) ou republication des images GHCR (mode prod).
Conflit de ports sur la machine hote	Echec du <code>up</code>	Liste des ports requis dans le <code>README.md</code> , remappage de la partie hote possible dans le compose.